Polling-Dependent NVML Energy Estimates on a Laptop GPU

Ishan Teredesai*, Tushar Rahangdale*, Karanjyot Singh†, and Ayush Pure†
*Department of Information Technology, Yeshwantrao Chavan College of Engineering (YCCE), Nagpur, India
teredesaishan@gmail.com, tusharrahangadle@gmail.com
†Department of Computer Science and Engineering, Yeshwantrao Chavan College of Engineering (YCCE), Nagpur, India
seemamission547@gmail.com, pureayush0@gmail.com

***Abstract*—GPU telemetry can change the apparent energy benefit of a software optimization. We examine this problem in a controlled case study on one NVIDIA GeForce RTX 4050 Laptop GPU, using 144 trials in four fresh process sessions within one sitting. During static rendering, increasing cumulative-energy polling from a nominal 1,000-ms interval to 25 ms raises the median counter-to-integrated-power ratio from 3.22 to 75.98. A no-launch idle control produces ratios of 152.90–198.22, while sustained rendering with boundary counter reads gives ratios near one. Exact frame reuse provides a software-optimization use case: the tested outputs remain correct and sparse-change validation reduces median frame time from 4.796 to 1.845 ms, but the two telemetry-derived saving estimates disagree in sign for 10 of 36 shared-baseline contrasts. This descriptive count measures sensitivity to estimator choice, not an error rate against ground truth. We provide the acquisition records, controls and reproducible analysis needed to inspect the discrepancy. The cause remains unresolved, including possible instrumentation error; neither telemetry path is independently calibrated. The practical contribution is a procedure for checking acquisition behavior before making energy claims about software changes.**

***Index Terms*—GPU energy, temporal reuse, energy measurement, sustainable software, reproducibility**

I. Introduction

Software engineers use energy measurements to choose implementations and evaluate optimizations. On a GPU, a shorter frame can leave more time waiting at the same service rate while uploads, synchronization and device context remain. An energy estimate therefore needs its own evaluation. It also depends on how the telemetry is collected: reading an exposed counter more often may change the reported result.

We study this acquisition problem on one laptop GPU. The central observation is a large dependence of cumulative-energy estimates on counter-read frequency during low-load execution. A no-launch control exposes the discrepancy without rendering work, whereas a sustained-load diagnostic gives much closer agreement between the counter and integrated power. A controlled frame-reuse benchmark connects these measurements to an actual implementation choice. Explicit tile dependencies let us check that policies produce the same tested output while changing the amount of rendering work.

The study asks three questions. *RQ1*: how do counter-read frequency and collection mode affect the energy estimates? *RQ2*: what do no-launch, copy-only and sustained-load controls reveal about the discrepancy? *RQ3*: how does estimator choice affect the apparent benefit of exact frame reuse?

We contribute a bounded empirical case study and a reproducible procedure for auditing GPU energy measurements. The completed study contains 144 trials across four process sessions on one laptop in one sitting. Its contribution beyond established polling-sensitivity results is the combination of a no-launch control, collection-mode comparisons, a sustained-load diagnostic and matched software-optimization contrasts. These controls expose which parts of an energy interpretation the measurements support. They do not identify the faulty layer: the Python acquisition code, NVML stack and device remain possible sources. The artifact makes that unresolved result inspectable and supports independent replication.

II. Related Work

a) GPU energy measurement: Burtscher, Zecena, and Zong document sampling and response anomalies in the K20 built-in sensor [1]. Yang, Adamek, and Armour characterize NVIDIA power sensing across GPU platforms and show that acquisition behavior matters for energy estimates [2]. These results motivate testing the sensor interface, but neither supplies a correction or accuracy bound for our laptop.

Dauner et al. examine sampling frequency for RAPL and NVML [3]. Their consumer-GPU results include cumulative-energy estimates below integrated power at short intervals. Our repeated-counter observations are above the integral, including at substantially longer nominal intervals. Polling sensitivity is therefore not a new general finding, and agreement on its existence does not establish a common mechanism. Our focus is how acquisition behavior affects a concrete reuse-policy evaluation and whether the same anomaly survives a no-launch control.

GPUWattch provides a hardware-validated architectural power model [4]. Jay et al. compare software power meters against external measurements and examine reporting scope and overhead [5]. PowerSensor3 supplies open instrumentation for measuring devices including GPUs [6]. These works illustrate complementary measurement approaches. Our study audits the software interface defined by NVIDIA's NVML documentation [7].

b) Experimental structure: Ardito et al. describe a documented process for measuring software energy [8]. Mytkowicz

et al. show how setup choices can bias performance measurements [9]. Kalibera and Jones distinguish variation across experimental levels when allocating repetitions [10]. We follow the practical implications of these works by retaining raw observations, recording the protocol, and separating frames from repeated process sessions.

c) Reusing rendering work: Scherzer et al. survey temporal-coherence methods and the conditions under which previously computed information remains useful [11]. Reverse reprojection caching retains shading calculations at visible surface points and introduces policies for refreshing cached entries [12]. Rendering Elimination detects redundant tiles through signatures of their inputs [13]. Temporally Adaptive Shading Reuse considers temporal coherence with perceptual constraints in real-time rendering and virtual reality [14].

These studies establish the prior basis for avoiding repeated shading. Our benchmark uses a narrower model: each tile depends on three state words, and unchanged dependencies must produce exactly the same pixels. It has no reprojection, perceptual tolerance, or cross-tile lighting. The cited systems provide prior context; our implemented baselines isolate validation, known invalidation and dispatch costs within the same renderer.

III. Controlled Software-Optimization Use Case

A. A restricted dependency model

The primary workload generates a 960×540 RGBA image in 48×48 tiles. There are 240 tiles, including a partial bottom row. Each tile contains a synthetic scene with six spheres, two lights, and four subpixel samples per pixel. We choose explicit, independent tile dependencies so that invalidation is inspectable and every policy can be checked against the same full-render output. No image is displayed. A common output-buffer copy represents a fixed device-side cost, rather than display presentation.

State consists of 65 unsigned 32-bit words. Tile i depends on global word 0 and local words

$$a_i = 1 + (i \bmod 64), \qquad b_i = 1 + ((13i + 7) \bmod 64). \quad (1)$$

The local indices are distinct because equality would require $12i + 7 \equiv 0 \pmod{64}$. For state s_t, define

$$d_{i,t} = \bigvee_{j \in \{0, a_i, b_i\}} [s_t[j] \neq s_{t-1}[j]]. \quad (2)$$

After an initial full rendering, recomputing tiles with $d_{i,t} = 1$ and retaining the others preserves the full-render result, provided the dependencies are complete and the execution is deterministic. This conditional argument explains the reuse rule. The implementation is checked separately through pixel comparisons.

B. Policies and change regimes

FULL shades the whole image every frame. VALIDATE uploads the current state, compares all 65 words with the previous state in a GPU kernel, and launches a full-grid render kernel whose unchanged pixels return early. DIRTY builds a dirty-tile mask from known changes on the CPU, uploads it, and uses the same full-grid early-return approach. COMPACT builds and uploads an active-tile list and launches only those tiles. It omits the render launch when no tile changes.

The dirty policies are useful baselines because the input generator already knows which state words changed. Their CPU bookkeeping and uploads are included in frame wall time. All four policies copy the complete image to a second device buffer and update the previous state each frame. The compact policy therefore still has nonzero cost when its render launch is absent.

The main study tests static inputs, sparse changes and all-changed inputs. Sparse changes independently modify each of the 64 local words with probability 0.02 per frame; global word 0 is unchanged. All-changed inputs modify every word. These regimes isolate no change, limited change and complete invalidation.

Every matched policy consumes the same precomputed warmup and measurement traces. Their SHA-256 values are checked within each block. Trace generation occurs outside measurement; CPU policy decisions and transfers remain inside. The trace represents controlled state transitions, not an empirical distribution of changes in games or user interfaces.

C. Correctness and lower-work controls

Correctness tests compare each reuse policy with FULL under retention, isolated changes and reversions of all 65 words, and randomized sequences. The first process tests both 960×540 and 961×541 grids and checks a shorter sequence at 1920×1080. Later processes repeat the short primary-grid check. A blind-stale implementation deliberately retains invalid output as a negative control. A final full-render comparison is also made after every measured trial, outside its energy window.

The copy control retains state uploads, CUDA events, the complete image copy, and the previous-state copy, but skips rendering on static inputs. The idle control retains the CUDA context and allocations and executes a 60-Hz host loop. It makes no CUDA launches, copies, event calls, or synchronization calls during warmup or measurement. It is application no-work, not whole-machine idle: telemetry and ordinary system processes may still run.

IV. Experimental Design and Analysis

A. Platform and acquisition design

The acquisition identifies a Lenovo 83DV, Intel Core i5-13450HX, approximately 16 GiB RAM, and GeForce RTX 4050 Laptop GPU with 6 GiB device memory. The log records NVIDIA driver 581.86, NVML 13.581.86, Python 3.14.5, and the Balanced Windows power scheme. Its Windows registry reports build 26300.9818 and DisplayVersion 26H2. Flight-channel fields are absent, so we do not infer a release channel. CUDA and NVML handles are matched by PCI bus identifier.

The embedded kernels were compiled with NVRTC 12.1.105 for `compute_52`, with fused multiply-add disabled.

TABLE I
COMPLETED STUDY DESIGN. COUNTS INCLUDE ALL FOUR SESSIONS. THE 48 MAIN TRIALS FORM 12 FOUR-POLICY BLOCKS.

Component	Trials	Window (s)
Policy comparison	48	20
Polling-rate sweep	52	15
Idle/copy controls	24	20
Identical-code A/A	8	20
Explicit power fields	8	20
Sustained load	4	15
Total	144	–

Their source and PTX digests are retained. The process inventory includes ordinary background applications and services; this is not a dedicated bare-metal test environment.

The main study completed all 144 planned trials in four fresh process sessions over 69.18 minutes. Table I gives the workload windows. Each session contributes one complete block for each main scenario and one repeat of every diagnostic condition. Main-policy order follows a four-period Williams design, balancing position and immediate predecessors across sessions. Other blocks are shuffled with recorded seeds. Scenario seeds change between sessions, while policies within a block share their trace. Sessions are separated by a scheduled 60-second pause. They are repeated executions in one sitting, not independent days or machines.

An earlier pilot, retained in the artifact, completed 88 of 306 planned trials before manual interruption. It contains unexplained pauses and incomplete planned ordering. It informed the acquisition design but is excluded from all tables, figures and policy estimates reported here. Its raw records and separate analysis remain available for audit.

B. Timing, monitoring and stopping

Each paced trial has a two-second pretrial idle interval, full initialization, a five-second policy-specific warmup, and its measured window. Absolute deadlines are spaced at $1/60$ s. Missing one deadline does not shift later deadlines. Host frame time includes bookkeeping, uploads, GPU work, copies, synchronization and event queries. Five CUDA events separately bracket upload, validation, rendering and copying. These are stream elapsed intervals [15]; host submission gaps can contribute, so they are not isolated shader instruction times.

Mode P polls ordinary NVML power plus temperature, clocks, performance state, throttle status and utilization. P+C additionally polls the cumulative energy counter. Mode B performs only the boundary counter reads during the workload; operating-state snapshots occur outside the measured window. Every mode reads cumulative energy at the workload boundaries. A common host-only watchdog checks for long scheduling gaps and AC loss without making CUDA or NVML calls.

The main trials use P at a nominal 100-ms interval. The polling sweep uses static FULL with nominal intervals of 25, 50, 100, 250, 500 and 1,000 ms under both P and P+C, plus B. Poll starts are randomized within $\pm 20\%$ of the nominal interval, with actual query times retained. Late scheduling does not generate a catch-up burst. The two explicit power-field conditions use ordinary P, either alone or with fields 185 (averaged power) and 186 (instantaneous power). Raw field types, return codes and timestamps are retained; these fields are not substituted into the primary estimator.

The sustained diagnostic renders static 1920×1080 images without a frame-rate cap for 15 seconds, following a five-second uncapped warmup. It runs last in each session and uses P. Its resolution, cadence and operating state differ from the main comparison. It tests agreement under another regime, not the energy benefit of reuse at matched service.

The main study has a fixed stopping rule for correctness or API failure, AC loss, temperature at least 85°C, watchdog gaps above five seconds, substantial clock discontinuity, unbracketed integration, or a workload overrun above two seconds. It does not stop for a favorable result. No such stop was recorded. Each session's watchdog event list is empty, all trial endpoints report AC power, and recorded telemetry temperatures span 42–58°C. These checks document observed conditions during the completed study.

C. Estimators and units of analysis

For cumulative readings C_0, C_1 in millijoules, we compute

$$\widehat{E}^C = (C_1 - C_0)/1000. \tag{3}$$

Each query records host timestamps before and after the call. Let u_0, u_1 be the midpoints of the two boundary queries. Ordinary power readings, converted to watts, are assigned to their own query midpoints and linearly interpolated as $\widetilde{P}(t)$. We then compute

$$\widehat{E}^P = \int_{u_0}^{u_1} \widetilde{P}(t)\,dt, \qquad R = \widehat{E}^C/\widehat{E}^P. \tag{4}$$

A post-workload tail brackets the endpoints. The artifact also contains integrals over the workload timestamps. Reported average power divides energy by the corresponding duration. Endpoint alignment removes one accounting mismatch; it does not establish the sensor's internal acquisition time.

NVML documents one-second averaging for ordinary power on Ampere, except GA100, and newer architectures [7]. Polling more often does not imply independent samples at that rate. Its stated Fermi/Kepler accuracy qualification is not an RTX 4050 accuracy bound. Neither estimator is an independent physical reference, and neither measures total-system energy. We do not subtract an assumed idle baseline.

For policy p and full-render baseline f in block b, the apparent saving is

$$S^X_{b,p} = 100\left(1 - \frac{\widehat{E}^X_{b,p}}{\widehat{E}^X_{b,f}}\right), \quad X \in \{C, P\}. \tag{5}$$

We report the median of the four paired percentages and their observed range. Opposite signs require $S^C S^P < 0$. The three reuse contrasts in a block share a baseline; the 36 contrasts are not 36 independent replications. The disagreement count describes sensitivity to estimator choice. It does not identify

TABLE II
POLLING SWEEP: MEDIANS ACROSS FOUR SESSIONS. ACHIEVED INTERVAL SUMMARIZES EACH P+C TRIAL'S MEDIAN QUERY SPACING. P AND P+C COLUMNS GIVE $R = \widehat{E}^C/\widehat{E}^P$. THE FIGURE SHOWS INDIVIDUAL RATIOS.

Nominal (ms)	Achieved (ms)	P: C/P	P+C: C/P
25	30.7	1.14	75.98
50	46.9	1.20	39.31
100	94.6	1.16	20.39
250	256.5	1.13	8.83
500	505.8	1.24	5.03
1000	1001.2	1.25	3.22

errors relative to physical truth or give the estimators equal authority. Timing tables summarize per-trial medians, with quartiles across the four trials. Ranges and quartiles are descriptive, not confidence intervals. Fits of counter increments against query count are also descriptive and were chosen after examining the sweep, without an inferential p-value.

V. RESULTS

A. Polling rate dominates the counter result

Figure 1(a) and Table II show the clearest result. Under P+C, the median ratio R is 75.98 at nominal 25 ms and 3.22 at 1,000 ms. All four sessions show the same ordering across the six rates. The measured work remains static full rendering at 60 Hz. In matched P trials, median ratios across the rates remain between 1.13 and 1.25.

The power-integral change between P+C and its matched P trial ranges from -2.00% to $+3.80\%$ across the 24 sweep pairs, with median $+0.43\%$. The much larger cumulative-counter change therefore depends strongly on the acquisition procedure even when this other telemetry path changes little.

Actual timing matters. Nominal 25-ms P+C polling has a median achieved interval of 30.7 ms when the four trial medians are summarized; scheduling variability makes the label different from exact periodic sampling. We therefore also fit additional reported counter energy, P+C minus matched P, against the number of continuous reads within the counter window. Separate six-point fits for the four sessions have slopes of 5.49–5.98 reported joules per read and R^2 between 0.9986 and 0.9999. These post-hoc slopes describe an association between total reported energy and read count. They neither establish a fixed jump on every read nor identify whether the effect arises in the acquisition program, driver or device. A direct C/NVML reproducer would test the Python acquisition path; no such hardware result is available in this study.

B. Idle, copying and sustained execution

The no-launch control strengthens the interpretation that avoided rendering work is not the source of the large counter increments. At nominal 100-ms P+C polling, its ratio is 152.90–198.22 across the four sessions (Table III). Dividing the counter increment by duration implies 245.78–308.75 W, whereas the simultaneous power integral implies 1.42–1.65 W. These are incompatible software estimates of the same interval, not a measurement of actual idle consumption at hundreds of watts.

The idle P+C power integral increases relative to P by 0.90% to 10.65% across the four pairs. The largest increase is appreciable, although still far smaller than the counter-to-integral discrepancy. Copy-only P+C ratios are 22.33–26.14. With boundary-only counter access, the implied counter power is much smaller for both controls. Read-frequency sensitivity therefore persists without shading and cannot be explained solely by how many tiles the renderer invalidates.

The sustained trials give a different observation. Integrated average power is 54.92–55.44 W and the ratio is 1.0097–1.0220. Every operating-state sample inside these four workload windows reports P0, with SM clocks of 2,685 or 2,700 MHz. Every one of the 9,588 samples inside main-trial windows reports P8 and 210 MHz. Sampled states do not prove a continuous state between queries. The sustained comparison changes resolution and cadence and uses P rather than P+C. It therefore does not test continuous counter polling in P0 or isolate performance state as a causal factor. It does show that the large low-load discrepancy is not reproduced in every tested workload regime.

C. Explicit power-field cross-checks

Both requested power fields succeed in all 1,062 snapshots in which they are requested, returning unsigned integer values. Relative to the ordinary-power integral over the same workload timestamps, the instantaneous-field integral differs by -0.18% to $+0.27\%$. The averaged-field integral is nearly identical in sessions 1 and 2, but is 4.89% and 4.74% lower in sessions 3 and 4. These four diagnostic trials use sequential calls. We retain the field-specific readings separately and keep ordinary power as the primary integral. Adding the field queries changes the ordinary-power integral by $+0.30\%$ to $+0.79\%$ in the four matched comparisons.

D. Completion, exact output and frame time

The completed study records 204,297 measured iterations. All 1,788 correctness comparisons and all 144 final-image checks pass; the blind-stale negative control produces mismatches. These checks cover the tested kernels, image dimensions and transitions, rather than arbitrary rendering workloads.

Recomputation from the archived raw counter values and power-query timestamps agrees with the recorded integrals within 5.7×10^{-13} J. Every completed workload duration is within 0.001077 s of its target. One deadline miss occurs in an idle-control host loop; none occurs in the 48 main trials. The acquisition therefore completes its intended design without concealing deadline failures or discarding an adverse trial.

Table IV shows where the performance benefit appears. For sparse changes, median frame wall time falls from 4.796 ms for FULL to 1.845 ms for VALIDATE. The median paired wall-time reduction is 61.97%. On static input, COMPACT reaches 1.533 ms while retaining common output and state-copy costs. Its nonzero render-event interval does not mean a shader was

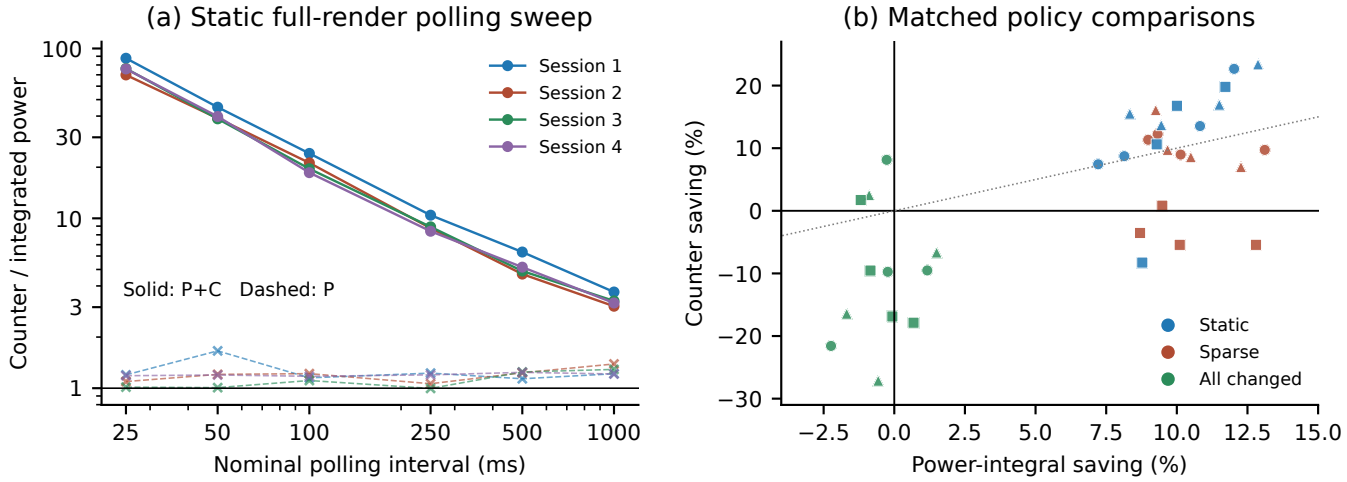


Fig. 1. Main-study observations. (a) Each line is one process session; solid lines include continuous counter reads (P+C), dashed lines use power/state polling with boundary counters (P). Axes are logarithmic. (b) All 36 policy contrasts, with circles for VALIDATE, squares for DIRTY, and triangles for COMPACT. Upper-left and lower-right quadrants indicate opposite signs. Points share block baselines and are not independent replications.

TABLE III

ABSOLUTE ESTIMATES AND CONTROLS. EACH ROW SUMMARIZES FOUR TRIALS AS MEDIAN [MINIMUM, MAXIMUM]. B HAS NO IN-WINDOW POWER POLLING, SO NO POWER INTEGRAL OR RATIO IS REPORTED. "FULL, STATIC" IS THE MAIN-POLICY BASELINE UNDER P; IDLE AND COPY CONTROLS USE NOMINAL 100-MS POLLING. SUSTAINED IS UNCAPPED 1920×1080 RENDERING UNDER P AND IS NOT A MATCHED OPTIMIZATION COMPARISON.

Workload	Mode	$\widehat{E}^C/\Delta t$ (W)	$\widehat{E}^P/\Delta t$ (W)	$\widehat{E}^C/\widehat{E}^P$
Full, static	P	3.68 [3.46, 4.00]	3.15 [2.57, 3.23]	1.22 [1.13, 1.35]
Idle	P	1.95 [1.88, 2.21]	1.49 [1.41, 1.58]	1.34 [1.19, 1.48]
Idle	P+C	280.11 [245.78, 308.75]	1.57 [1.42, 1.65]	184.39 [152.90, 198.22]
Idle	B	2.06 [1.82, 2.18]	–	–
Copy	P	3.05 [2.56, 3.35]	2.84 [2.19, 2.97]	1.13 [1.01, 1.16]
Copy	P+C	65.23 [58.76, 66.04]	2.82 [2.25, 2.96]	23.16 [22.33, 26.14]
Copy	B	3.39 [3.07, 4.10]	–	–
Sustained	P	55.95 [55.45, 56.58]	55.30 [54.92, 55.44]	1.01 [1.01, 1.02]

launched; the events can include submission gaps even when the render launch is omitted.

The CPU-known dirty policy is not uniformly slower than GPU validation. In the new sparse blocks, its median wall time is 1.748 ms, but its across-trial spread is larger. When every word changes, the reuse policies perform essentially the full rendering path plus their bookkeeping, with median wall times around 4.89–4.91 ms versus 4.839 ms for FULL. These observations argue for reporting several relevant baselines and their spread rather than selecting one favorable implementation comparison.

E. Policy comparisons depend on the estimator

Table V reports all nine scenario/policy cells, each with four paired blocks. For sparse VALIDATE, the counter and integral agree in direction in all four blocks, with medians of 10.54% and 9.73%. These are apparent savings under the specified acquisition procedure.

Across the main study, signs disagree in 10 of 36 contrasts. Six disagreements occur in the all-changed regime, where integrated-power effects are close to zero. Excluding that regime leaves four disagreements in 24 contrasts: three sparse DIRTY comparisons and one static DIRTY comparison. Seven of the ten disagreements have both absolute effects above 1%; three exceed 5%. All-changed is a useful low-benefit comparison, but not an identical-code null because the policies still execute different bookkeeping.

The magnitude is modest on the power-integral path. Static and sparse reuse cells have median apparent savings between 9.48% and 10.48%, corresponding to median paired average-power differences of roughly 0.29–0.32 W. Static FULL uses a median 52.42 mJ per frame from the workload integral, compared with 47.21 mJ for COMPACT.

The four adjacent A/A comparisons give counter apparent savings from -4.55% to $+1.32\%$, and power-integral values from -0.83% to $+0.95\%$. Their sample standard deviations are 2.60 and 0.76 percentage points. Four pairs do not define a universal noise floor. In particular, the small differences among reuse policies do not establish an energy ranking or equivalence.

VI. IMPLICATIONS FOR GREEN SOFTWARE ENGINEERING

A software-energy study should check its acquisition procedure before ranking implementations. On this configuration,

TABLE IV

MAIN-STUDY TIMING AND ABSOLUTE POWER ESTIMATES. WALL TIME IS THE MEDIAN OF FOUR TRIAL MEDIANS, WITH THEIR INTERQUARTILE RANGE. OTHER COLUMNS ARE MEDIANS OF TRIAL SUMMARIES. GPU, RENDER AND COPY VALUES ARE CUDA EVENT INTERVALS; THEIR MARGINAL MEDIANS NEED NOT ADD. $\widehat{P}$ USES THE COUNTER-ALIGNED POWER INTEGRAL.

Scenario	Policy	Wall time [Q1, Q3] (ms)	GPU (ms)	Render (ms)	Copy (ms)	$\widehat{P}$ (W)
Static	Full	4.703 [4.699, 4.725]	3.609	3.420	0.068	3.145
	Validate	1.782 [1.727, 1.871]	0.591	0.218	0.069	2.861
	Dirty	1.923 [1.841, 2.029]	0.571	0.261	0.069	2.849
	Compact	1.533 [1.531, 1.556]	0.323	0.014	0.122	2.833
Sparse	Full	4.796 [4.712, 4.874]	3.634	3.404	0.068	3.138
	Validate	1.845 [1.785, 1.904]	0.719	0.366	0.068	2.851
	Dirty	1.748 [1.476, 2.019]	0.601	0.369	0.068	2.853
	Compact	1.871 [1.786, 1.952]	0.624	0.283	0.069	2.841
All changed	Full	4.839 [4.783, 4.866]	3.631	3.402	0.068	3.145
	Validate	4.909 [4.884, 4.930]	3.742	3.337	0.068	3.131
	Dirty	4.893 [4.875, 4.913]	3.642	3.412	0.068	3.153
	Compact	4.892 [4.849, 4.915]	3.603	3.368	0.068	3.135

TABLE V

PAIRED APPARENT ENERGY SAVINGS AGAINST FULL IN THE MAIN STUDY. ENTRIES ARE MEDIANS [MINIMUM, MAXIMUM] OVER FOUR SESSION/SEED BLOCKS. "OPPOSITE" COUNTS COUNTER/INTEGRAL SIGN DISAGREEMENTS. EACH BLOCK SHARES ONE BASELINE ACROSS THREE REUSE CONTRASTS.

Scenario	Policy	Counter saving (%)	Power-integral saving (%)	Opposite
Static	Validate	11.13 [7.43, 22.67]	9.48 [7.22, 12.03]	0/4
	Dirty	13.69 [-8.27, 19.79]	9.65 [8.78, 11.72]	1/4
	Compact	16.23 [13.69, 23.39]	10.48 [8.34, 12.87]	0/4
Sparse	Validate	10.54 [8.98, 12.25]	9.73 [8.99, 13.11]	0/4
	Dirty	-4.49 [-5.44, 0.82]	9.79 [8.70, 12.80]	3/4
	Compact	9.15 [6.99, 16.09]	10.08 [9.25, 12.27]	0/4
All changed	Validate	-9.63 [-21.58, 8.14]	-0.25 [-2.24, 1.17]	2/4
	Dirty	-13.21 [-17.87, 1.72]	-0.46 [-1.19, 0.68]	2/4
	Compact	-11.55 [-27.13, 2.52]	-0.73 [-1.68, 1.50]	2/4

TABLE VI

PRACTICAL CHECKS BEFORE INTERPRETING GPU ENERGY ESTIMATES.

Check	What to report or compare
Acquisition	Compare boundary-only and repeated counter access at multiple rates; retain actual query times and raw return values.
Work removed	Include no-launch and copy-only controls; report how sustained execution changes cadence and monitoring mode.
Effect size	Report paired joules, watts and frame times alongside A/A variation; identify shared baselines and process sessions.
Suspect readings	Cross-check a minimal client and another device or driver; use a suitable external reference to assess physical accuracy.

the counter change associated with collection mode is much larger than the roughly 0.3-W paired difference reported by integrated power for reuse. Reporting only a percentage saving would hide that difference in scale.

Table VI summarizes checks motivated by the case study. They are diagnostic steps rather than calibrated settings. The tested data do not establish a universally safe polling interval, and agreement between two NVML paths is not external validation. A separate client can help test instrumentation, while physical-energy claims require a reference measurement with a compatible scope.

The reuse case illustrates why timing and energy need separate evidence. The tested policies preserve the output and shorten frames for static and sparse inputs. Their energy interpretation remains dependent on the telemetry path. The artifact retains raw records, source and protocol identities, trial-level results and analysis so that these distinctions can be inspected or challenged on another configuration.

VII. THREATS TO VALIDITY

a) Measurement validity: Both estimators come from the same exposed NVML stack. Agreement can reflect shared error, and disagreement cannot identify which value is physically accurate. No external GPU or whole-system meter was used; the estimates do not measure battery energy or establish calibrated physical savings. The acquisition uses the documented unsigned 64-bit counter output and converts millijoules to joules once. Energy is the difference between boundary readings, not a sum of cumulative counter values. Recomputing the archived arithmetic checks accounting consistency, but cannot validate the acquisition path itself. No C/NVML reproducer, separate acquisition client, second GPU or alternate-driver experiment was run. An instrumentation bug remains a live explanation alongside driver or device behavior; the responsible layer is unresolved. Multi-field queries are sequential; query timestamps and field timestamps do not establish independent sensor resolution.

b) Repetition and dependence: The main study contains four sessions on one laptop in one sitting. The main analysis has four blocks per scenario, with a shared baseline within each block. The interrupted pilot is excluded from the reported estimates. We report observed ranges rather than population confidence intervals. The polling fits are descriptive, and A/A spread is not an equivalence margin. No formal power calculation supports an energy-equivalence claim.

c) Operating conditions: AC endpoints, temperature checks and an empty watchdog event log do not eliminate background interference or establish continuous operating state. The process inventory includes other applications and services. Even under the same sampled P8 label, static full-render integrated average power varies from 2.57 to 3.24 W across sessions. This spread exceeds the roughly 0.3-W paired reuse effect, motivating within-block comparisons and limiting claims about stability across sessions. The idle control retains a GPU context, and the sustained test changes several factors at once. Those comparisons support a bounded case analysis, not causal identification of a power-state mechanism.

d) Workload and instrumentation: Independent tiles, simple state dependencies and synthetic changes make exact reuse easy to inspect but limit realism. The workload excludes display presentation, complex synchronization, CPU/system energy, and production rendering effects. Event instrumentation and Python/driver overhead contribute to the timed path. Correctness is established only for the tested kernels, dimensions and sequences. Generalization requires other devices, drivers, days and application workloads, followed by independent measurement if physical savings are claimed.

VIII. Future work

This is a bounded, reproducible case study of NVML acquisition behavior on one laptop GPU. We do not claim a general NVML/GPU defect or calibrated physical energy savings. Our contribution is a set of acquisition checks and controls that reveal when energy-based software-optimization claims may be estimator-dependent. Independent validation using another acquisition client, another device, another driver, and a physical reference remains necessary future work.

IX. Conclusion

On the tested laptop, cumulative-energy estimates vary strongly with counter-read frequency during low-load execution. The discrepancy persists in a no-launch control and is much smaller during the sustained-load diagnostic under a different monitoring mode. These observations characterize the recorded acquisition path; they leave its mechanism and physical accuracy unresolved.

Exact frame reuse supplies a concrete software-engineering consequence: correct tested output and shorter frame times coexist with estimator-dependent energy conclusions. The useful outcome is a reproducible set of acquisition checks and controls. Independent client, device and physical-measurement validation are the next steps needed to distinguish an instrumentation problem from a platform-level measurement limitation.

Supplementary Material

The benchmark, raw measurements and reproduction instructions are available at https://github.com/titcsl/reuse-model. The repository organizes the benchmark source, acquisition records, protocols, analysis code and derived results in separate directories. Both original CSVs are distributed with lossless gzip compression; their decompressed contents are checked against the acquisition hashes. Running `python reproduce.py` verifies the inputs and rebuilds the tables and figures without a GPU. The `restore_executed.py` utility recovers the exact program embedded in each acquisition. Source provenance and instructions for a separate hardware replication are included.

Acknowledgment

OpenAI ChatGPT assisted with code analysis, testing support, and some drafting and revision of the manuscript. Numerical results were computed from the recorded acquisition data.

References

[1] M. Burtscher, I. Zecena, and Z. Zong, "Measuring GPU Power with the K20 Built-in Sensor," in *Proc. Seventh Workshop on General Purpose Processing Using GPUs*, pp. 28–36, 2014. [Online]. Available: https://userweb.cs.txstate.edu/~burtscher/papers/gpgpu14.pdf

[2] Z. Yang, K. Adamek, and W. Armour, "Accurate and Convenient Energy Measurements for GPUs: A Detailed Study of NVIDIA GPU's Built-In Power Sensor," in *Proc. SC*, 2024. [Online]. Available: https://sc24.supercomputing.org/proceedings/tech_paper/tech_paper_pages/pap544.html

[3] M. Dauner, M. Steinberg, A. Brunnert, B. Schicker, and B. Zönnchen, "Evaluating the Influence of Measurement Frequency on Energy Readings Using Intel RAPL and NVIDIA NVML," *ACM SIGENERGY Energy Informatics Review*, vol. 6, no. 2, Jul. 2026. [Online]. Available: https://hotcarbon.org/assets/2026/paper-46.pdf

[4] J. Leng, T. Hetherington, A. ElTantawy, S. Gilani, N. S. Kim, T. M. Aamodt, and V. J. Reddi, "GPUWattch: Enabling Energy Optimizations in GPGPUs," in *Proc. Int. Symp. Computer Architecture (ISCA)*, 2013, doi: 10.1145/2485922.2485964.

[5] M. Jay, V. Ostapenco, L. Lefèvre, D. Trystram, A.-C. Orgerie, and B. Fichel, "An Experimental Comparison of Software-Based Power Meters: Focus on CPU and GPU," in *Proc. IEEE/ACM Int. Symp. Cluster, Cloud and Internet Computing (CCGrid)*, 2023, doi: 10.1109/CCGrid57682.2023.00020.

[6] S. van der Vlugt, L. Oostrum, G. Schoonderbeek, B. van Werkhoven, B. Veenboer, K. Doekemeijer, and J. W. Romein, "PowerSensor3: A Fast and Accurate Open Source Power Measurement Tool," in *Proc. IEEE Int. Symp. Performance Analysis of Systems and Software (ISPASS)*, pp. 213–226, 2025, doi: 10.1109/ISPASS64960.2025.00028.

[7] NVIDIA, "NVML API Reference Guide: Device Queries." Accessed: Nov. 6, 2026. [Online]. Available: https://docs.nvidia.com/deploy/nvml-api/latest/api/group__nvmlDeviceQueries.html

[8] L. Ardito, R. Coppola, M. Morisio, and M. Torchiano, "Methodological Guidelines for Measuring Energy Consumption of Software Applications," *Scientific Programming*, vol. 2019, Art. no. 5284645, 2019, doi: 10.1155/2019/5284645.

[9] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing Wrong Data Without Doing Anything Obviously Wrong!" in *Proc. Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 265–276, 2009, doi: 10.1145/1508244.1508275.

[10] T. Kalibera and R. Jones, "Rigorous Benchmarking in Reasonable Time," in *Proc. ACM SIGPLAN Int. Symp. Memory Management (ISMM)*, 2013, doi: 10.1145/2464157.2464160.

[11] D. Scherzer, L. Yang, O. Mattausch, D. Nehab, P. V. Sander, M. Wimmer, and E. Eisemann, "A Survey on Temporal Coherence Methods in Real-Time Rendering," in *Eurographics 2011 State of the Art Reports*, pp. 101–126, 2011, doi: 10.2312/EG2011/stars/101-126.

[12] D. Nehab, P. V. Sander, J. Lawrence, N. Tatarchuk, and J. R. Isidoro, "Accelerating Real-Time Shading with Reverse Reprojection Caching," in *Proc. ACM SIGGRAPH/Eurographics Symp. Graphics Hardware*, pp. 25–35, 2007. [Online]. Available: https://gfx.cs.princeton.edu/gfx/pubs/Nehab_2007_ARS/index.php

[13] M. Anglada, E. de Lucas, J.-M. Parcerisa, J. L. Aragón, P. Marcuello, and A. González, "Rendering Elimination: Early Discard of Redundant Tiles in the Graphics Pipeline," in *Proc. IEEE Int. Symp. High Performance Computer Architecture (HPCA)*, 2019, doi: 10.1109/HPCA.2019.00014.

[14] J. H. Mueller, T. Neff, P. Voglreiter, M. Steinberger, and D. Schmalstieg, "Temporally Adaptive Shading Reuse for Real-Time Rendering and Virtual Reality," *ACM Trans. Graph.*, vol. 40, no. 2, Art. no. 11, 2021, doi: 10.1145/3446790.

[15] NVIDIA, "CUDA Driver API Reference Manual: Event Management." Accessed: Nov. 2, 2026. [Online]. Available: https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__EVENT.html